→ memory Implementation of a list (mutable)

Python learning tool
All features
mem represent
All modules

All errors

x = (1, 2, 3)

x(1) = 100 — can't assign to a function call

x[1] = 100 → doesn't support item assignment

x.append (100)

↓

tuple has no attribute append

Based on order of Insertion — Indexing

7 Bits — ASCII encoding — Integer

+ - new list

+= → existing list

y = x[:] — creating new list

→ File Formats

PPPA156

→ Python is user friendly

→ Qv.

15/07/2026

Functions – reusability

Exceptions – error Handling
- → memory error
- → dangling point error
- → divide by zero
- → Index out of Bound

→ Key word – reserved word

```
def function_name (parameters):
    statements
    return [expression]
```

→ 1 Array → 1 address – Base address

So, return type ← pointer type

→ ** – address of an address
" double pointer "

→ A function is a block of code i.e reusable code and performs a task.

```
def sum(a, b):
    c = a + b
    print("sum = ", c)
```

sum(1,2) → function call
sum = 3 → output

Types of Passing Arguments:

1. Default Arguments ← setting a value to an argument while creating the columns in RDBMS
2. Keyword Arguments → order is not required can be called in any order "name = value"
3. Variable-length → argumentname
   (normal arg, (remaining arg is form of tuple))

1. Default Arguments: → Setting a value to the argument

```
def display (name, age = 35) :
    Print ("Name : ", name)
    Print ("Age : ", age)
display (name = "balu")
```

→ If we don't pass any value — the default argument will be taken

Keyword Argument

2. Keyword Arguments:

order is not required

```
def display (name, age):
    Print ("Name : ", name)
    Print ("Age : ", age)
display (age = 20, name = "balu")
```

Keyword Arg = names of the arguments

3. Variable length Arguments:

```
def function_name (argc, *argv):
    statements
    return (exp)
```

```
def disp (arg1, *arg2) :
    print
```

disp (3, 1, 2, 3)

no. of args Passed — arg2

16/07/20

Character Encoding

1 Byte : 8 bits

2 Byte : 16 bits

2^{16}

4-Byte :

ASCII

7 Bit encoding

$2^7 \Rightarrow 128$ chars

→ Increase the size of representing a character

→ Unicode → Introduced by "IBM"

Lambda Functions

→ g = lambda x : x ** 2 — expression

(x : Input)

lambda ↓ Keyword

→ doesn't have return sta

define a function with variable length argument,

One of the argument should be iterable and check if the valid age or not.

Legal age ⇒ 17

using Lambda, map function

```
def validate_age (*ages):
    return (list (map (lambda
        age : 17 < age <= 25 , ages))

res = validate_age (16, 17, 25, 27)
```

→ variable length → tuple
→ If typecasted → List

map → All elements

→ Filter — Applies the condition to those values that satisfies the condition → Based on condition

filter (lambda input : condition , iterable)

map (lambda input :

list (filter (function, iterable)) ✓

list (map (function, iterable)) ✓

1 * 2 * 3 * 4 * 5
2
20 * 6
120

→ Sorted

→ Reduce → iterable ✓

reduce(x, y: x * y, lst)

→ multiple

→ lst = [1, 2, 3, 4, 5]

→ reduce (lambda x, y : x * y, lst)

argument 1

argument 2

Iteratively and get output as a single

Sorted → creates a new object

22/9/2026

Multiprocessing

```
import multiprocessing
import time
for _ in range(10):
    P = multiprocessing.Process(fn)
    P.start()
    P.join()
```

lst = [ ]

lst.append(P)

All the processes that have been started

```
P1 = multiprocessing.Process(fn)
P2 = multiprocessing.Process(fn)
P1.start()
P1.join()
```

→ Tasks ✓

→ Parallel Processing

→ 3 functions -

- Process()
- start()
- join()

```
for i in lst:
    i.join()
```

→ function executes → when start() is called

```
import multiprocessing
def fn():
    time.sleep(1)
    print("Done")
P1 = multiprocessing.Process(target = fn)
```

Pass the argument

P1.start() → start the process

P1.join() → wait till the task is completed

Print('End')

P2 =

P3 =

P4 =

Print(f'Finished in {round(finish - start, 2)} Seconds')

```
lst = [ ]
for _ in range(10):
    p = multiprocessing.Process(target=fn)
    p.start()
    lst.append(p)
for p in lst:
    p.join()
```

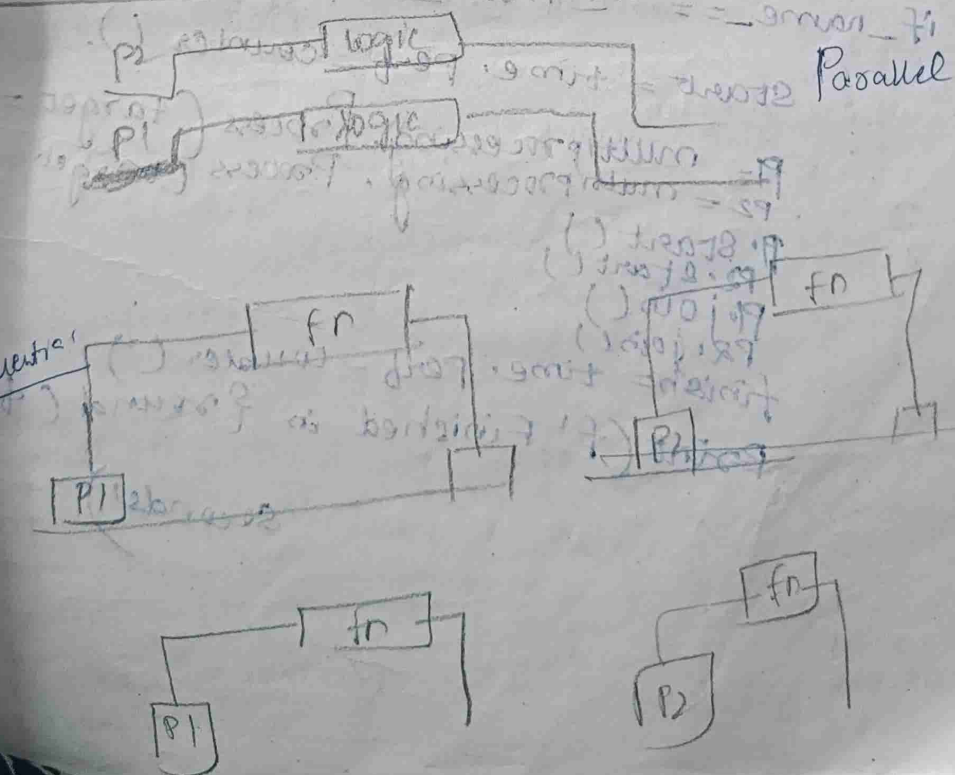
```
start = time.perf_counter()
finish = time.perf_counter()
```

2sec – 2 things

```
import time
import multiprocessing

start = time.perf_counter() → captures the time at that particular point

def do_something():
    time.sleep(1)
    Print('Done Sleeping...')
```



24/07/2026

- shallow copy
- deep copy
- sorted
- List unpacking
- List comprehension

Tracing a for loop :-

```
i = 5
for i in range(10):
    i = i**2
```

Black Box

→ input
→ output

Elevator Pitch

```
import multiprocessing
import time
start = time.perf_counter()

def do_something():
    print('Sleeping 1 second')
    time.sleep(1)
    print('Done Sleeping')

if __name__ == '__main__':
    start = time.perf_counter()
    p1 = multiprocessing.Process(target=do_something)
    p2 = multiprocessing.Process(target=do_something)
    p1.start()
    p2.start()
    p1.join()
    p2.join()
    finish = time.perf_counter()
    print(f'Finished in {round(finish-start, 2)} seconds')
```

"Concurrent.features Module"

a

Concurrent "futures" — module for multiprocessing

→ serialized object

29/07/2026

Itertools

→ Iterable - entire collection of a datastructure

→ Iteration -

→ Iterator - the variable that a programmer hold to control the executa

↓

loop counter variable

Strasin — $O(2^{81})$

Strasin

↓

'n' is larger

Iterable
lists
tuples
dict
set

→ Infinite
→ Combinatoric
→ Terminating

→ Rewrite + using these

↓

Leetcode

args = (2,)

→ operator.mul ✓

→ print execution time

↓

Other functions using operator module

→ import itertools

→ import operator

↓ iter

nums = [10, 20, -30]

i = iter(nums)

next(i)

→ index-iterator

__iter__

__next__

dir

Explain — 5 Exceptions faced so far

and put the code in try - catch

→ i = 10
→ While True

→ Finally

→ Infinite Iterators:

1. count (start, step)

Value
Dangling Point
Divide by zero

iterable
↑
→ Inbuilt iterator
→ dund

count

```
import itertools
for i in itertools.count(5, 5):
    if i == 35:
        break
    else:
        print(i, end = " ")
```

5 → start
Increase by 5

```
import itertools.
count = 0
for i in itertools.cycle('AB'):
    if count > 7:
        break
    else:
        print(i, end = " ")
        count += 1
```

iter
count
cycle
repeat

Infinite Iterators

Explain 5 Exceptions that you have faced so far and put the code in try-catch

→ i = 10
→ While True

→ Finally

→ Infinite Iterators:

1. count (start, step)

Value
Dangling Point
Divide by Zero

iterable
↑
→ Inbuilt iterator
→ dund

count

```
import itertools
for i in itertools.count(5, 5):
    if i == 35:
        break
    else:
        print(i, end = " ")
```

(5 → start; 5 → Increase by 5)

```
import itertools.
count = 0
for i in itertools.cycle('AB'):
    if count > 7:
        break
    else:
        print(i, end = " ")
        count += 1
```

iter
count
cycle
repeat

Infinite Iterators

Combinatoric Iterators

→ Product

→ Permutation — order matters — $AB \neq BA$ — more objects

→ Combination — order doesn't matter
↓
$AB = BA$

— Image Augmentable
↓
generating more images

→ flipping
→ Rotating
→ Inverse
↓
negative.

→ Zoom-In
→ Spurious Tuples

— Terminating Iterators
- → accumulate
- → compress
- → dropwhile
- → filterfalse
- → takewhile
- → chain

Infinite
- Count
- cycle
- repeat

Combinatoric
- Permutation
- combination
- Product

→ Unique Module :-
→ Developer team ← finance, Healthcare } domain sp
→ Very Very Unique
→ optimization → 10 tasks.

10 functions

→ Who contributed
→ Inp | outp | comments | Exceptions |
what is it

→ Terminating — short input sequence
function — o/p

accumulate (iter, fun)

Print(list(itertools.accumulate(list1)))

```
import itertools
import operator
list1 = [1,4,5,7]
print(list(itertools.accumulate(list1)))
print(list(itertools.accumulate(list1, operator.mul)))
```

addition

Chain: — joins all the elements in the list

```
print(list(itertools.chain([1,2,3])))
```

compress(iter, selector) :—

```
print(list(itertools.compress('GEEKSFORGEEKS',
    [1,0,0,0,0,1,0,0,1,0,0,1])))
```

SOUMYA

(fun, sec)

dropwhile :— starts printing once the false start

```
li = [2,4,5,7,8]
list(itertools.dropwhile(lambda x: x%2==0, li))
```

filterfalse:

```
print(list(itertools.filterfalse(lambda x: x%2==0, li)))
```

starmap — only tuple

```
li = [(1,10,5), (8,41)]
```

(itertools.starmap(min, li))

1st	1	1
2nd	1	1
3rd	2	4
4th	3	9
5th	5	25

Generator

produce

Program

def

for

possible

o/p

	x	y
	0	
1st	1	
2nd	1	
3rd	2	
4th	3	
5th	5	

1^2+1^2+2

3^2

Generators → "yield" — don't use return

↓

produce series of results

→ How state of an object is maintained

→ yield

```
def generator_my(n):
    yield something
```

Program

```
def my_generator(n):
    value = 0
    while value < n:
        yield value
        value += 1

for value in my_generator(3):
    print(value)
```

O/P :-
0
1
2

possible to nest these objects → generator objects

```
def fibanocci_numbers(nums):
    x, y = 0, 1
    for _ in range(nums):
        x, y = y, x+y
        yield x

def square(nums):
    for num in nums:
        yield num**2

print(sum(square(fibanocci_numbers(5))))
```

O/P

	x	y
	0	1
1st	1	1
2nd	1	2
3rd	2	3
4th	3	5
5th	5	8

$x, y = y, x+y$

$1^2+1^2+2^2+3^2+5^2$

$1+1+4+9+25$

```
import memory_profiler
import random
import time

names = ['Harsh', 'Abhijit', 'Soumya', ---, ]
experience = ['AWS', 'Deep', ---]
Print('Memory (Before): {}'.format
      (memory_profiling.memory_usage
```

→ To initialize values for instance variables > Constructor

self → Instance for itself

S1 = Student

```
def __init__(self, name, cgpa):
```

S1 = Student('arpan', cgpa)

Default copy Parameterized } Constructor

to copy the values

```
class Person:
    def __str__(self):
        return f'{self.name} {self.age}'

p1 = Person('Shibu', '25')
print(p1)
```

To Print the object dir

init → initialise the instance

copy constructor → copy the variable and values from one object to another

copy constructor:

```
def __copy__(self):
    return Person(self.name, self.age)

p2 = p1.__copy__()
```

o/p:- ('Shibu', '25')

Method Overloading — no need to define the method everytime

```
class MathOperation:
    def add(self, a, b=None, c=None):
        if b is not None and c is not None:
            return a+b+c
        elif b is not None:
            return a+b
        else:
            return a

math_obj = MathOperations()
res1 = math_obj.add(5)        # 5
res2 = math_obj.add(5,10)     # 15
res3 = math_obj.add(5,10,15)  # 30
```

"overload"

Print(res1)
Print(res2)
Print(res3)

9/08/2026

Threading : (I/O bound tasks)

3.2 Pytho

- Running a simple function
- Running it multiple times
- Using thread/process
- Special thread methods
- creating threads in a loop
  - ↳ dont make it sequential
- Passing arguments to thread()
- threadPoolExecutor() method
- submit() method
- result() method → you can have the result later
- Loop it / list comprehension.

---

context manager

with → no need to close the explicitly

```
with concurrent.futures.ThreadPoolExecutor()
as executor:
    f1 = executor.submit(do_something, 15)
    print(f1.result())
```

result() → to fetch the result

f1 → thread created using ThreadPool Executor.

```
with concurrent.futures.ThreadPoolExecutor() as
    executor:

results =
```

with concurrent.futures.ThreadPoolExecutor() as executor:

f1 = executor.submit(do_something, 1.5)

Keeps the task list of readily

holds future object

f2 = executor.submit(do_something, 1.5)

print(f1.result())

to fetch the result

print(f2.result())

"as_completed"

submit → Future object

map → holds the output only

~~lst = [ ]~~

with concurrent.futures.ThreadPoolExecutor() as executor:

res = [executor.submit(do_something, 1.5) for _ in range(2)]

for f in concurrent.futures.as_completed(res):

print(f.result())

Total "5"

secs = [5, 4, 3, 2, 1]

res = executor.map(do_something, secs)

class obj for ThreadPoolExecutor

Sleep:-

1st th = 5 sec
2nd th = 4 sec
3rd th = 1 sec

→ requests — developer

→ urls = [———]

→ • get the url ✓
  • content of the img ✓
  • split → ✓
  • Print the msg → ✓

{ "Unsplash" : ——— , "imgur" : ——— } photo
→ thug

Concurrent futures → ✓

function ✓

threadpool/Executor —

List ✓

✓ PriorityQueue } OS — Race condition
✓ Lock variable }



04/09/2020

Numpy (2005) — Travis Oliphant

(Numerical Python)

→ Array → continuous memory allocation

→ Spatial locality ← Keeping things together which are accessed together and accessed again & again
Temporal locality of reference

→ Numpy Arrays — 50 times faster than lists.

→ Locality of reference ✓

✗ creating Array objects ✓

1. creating array object from list

```
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

o/p:
[1 2 3 4 5]

2. creating array from tuple

```
arr = np.array((1, 2, 3, 4, 5))
print(arr)
```

o/p: [1 2 3 4 5]

✗ 3. Dimensions in Array ✓

0-D Array

```
arr = np.array(42)
```

o/p: 42

1-D array

```
arr = np.array([1, 2, 3, 4, 5])
```

[[1 2 3 4]
[3 4 5 6]]

Props of multidimensional Array

size - no. of elements

shape - no. of rows, n columns

3-D Array :-

```
arr = np.array([[[1,2,3],[4,5,6]],
                [[7,8,9],[10,11,12]]])
```

nested lists

o/p:

```
[[[1,2,3],
  [4,5,6]
 ],
 [[7,8,9],
  [10,11,12]
 ]
]
```

shape : (2, 2, 3)

size : 12

ndim : 3

rows & columns

Basic characteristics :

```
arr = np.array([[1,2,3],[4,2,5]])
Print(arr.ndim)   — built in Attribute
```

o/p: 2

```
Print(arr.shape)  —— (2,3)
Print(arr.size)   —— 6
Print(arr.dtype)  —— int64
```

Array creation

random — Blw 0 and 1
randint

c = np.zeroes(3,4) → $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

c = np.ones(2,3)

c = np.full((3,3),6)

e = np.random.random((2,2))

f = np.arange( ) ✓

g = np.linspace(0,5,10) → 10 values

range: 0 to 5

5 | 9

[0, 0.5555556

reshape

arr = np.array([[1,2,3,4],[5,2,4,2],[1,2,0

new = arr.reshape(2,2,3)

Flatten Array

arr = np.array([[1,2,3],[4,5,6]])

flarr = arr.flatten()

16/10/2026

Data Wrangling

Naman Shrivastav

↓

Cleaning data

→ bias

→ missing values ✓

→ updating ✓

→ Duplicates ✓

missing principles ✗ ✗

Join

↑

"B/w 2 tables"

→ Adv Python Portfolio

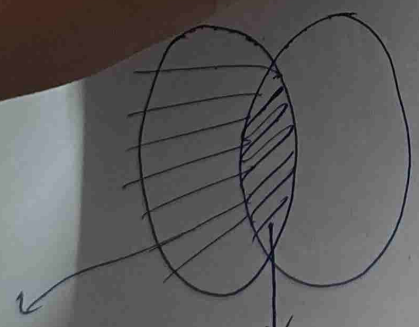
↓

Based on

→ Assessments

→ Extra readings

→ Answers given



Left outer join

Inner join

↓

common column ID

Cartesian product

↓

→ Spurious records "not existing"

typeerplane - DB
distance b/w two

Design { IOT Based Support System for impaired person

% of people affected by Impairment

AI to :

Representation Bias

How do you prepare this data for classification.

Bias - created data

Data Wrangling
→ Missing data
→ Duplicates

Sundarbans
Human Animal Conflict

data

Join
< B/w 2 table

10/2026